Finite-Ones Compactness and Resource-Existence Decidability for Epsilon-Free Deterministic Pushdown Automata

Alp Eren Bütün
Independent Researcher, Türkiye
ORCID: 0009-0001-2479-6037
GitHub: https://github.com/alperenbutun

October 2026

Abstract

For the binary deterministic pushdown languages $B_{m,k}$, we decide whether an equivalent *epsilon-free* recognizer exists with at most q control states and g physical stack symbols, even when the length of the stack word written by a transition is unrestricted. The result holds uniformly for all $m \geq 2$, $k \geq 1$, and $q, g \geq 1$ under a singleton initial stack and final-state-plus-empty-stack acceptance. The essential mechanism is a *finite-ones calibration*: every live source prefix with at most H input ones has an accepted zero-only completion of bounded length. This forces a horizon-dependent bound on the physical stacks of *all* correct epsilon-free competitors, allowing deletion of long transition rules without changing finite-horizon behavior. A deletion-only compactness theorem, together with decidability of deterministic pushdown equivalence, yields a terminating yes/no procedure. Every negative resource instance additionally admits a *finite, mechanically verifiable refutation packet* of input words, and the method extends to effectively zero-drain-calibrated target languages. We give a small exact finite-horizon certificate and an additional $B_{3,1}$ computational Pareto case study; the latter's numerical frontier claim is explicitly conditional on the full validity of its archived finite case-reduction certificates. The general decidability and finite-refutation theorems do not depend on those numerical certificates. No conclusion about fixed-resource existence for epsilon-enabled competitors is asserted.

Keywords: deterministic pushdown automata; resource-existence decidability; finite-ones compactness; stack alphabet; finite negative certificates; realtime recognition.
2020 Mathematics Subject Classification: Primary 68Q45; Secondary 68Q25.

1 Introduction

A resource question for a deterministic pushdown language asks whether it has a recognizer within prescribed limits on the number of control states and physical stack letters. Fixing these two finite resources does *not* make an exhaustive search automatically finite, because a transition may replace the top stack symbol with an arbitrarily long but finite word. One cannot solve the resource-existence problem merely by searching transition tables up to some *a priori* arbitrary replacement bound.

For the binary witnesses $B_{m,k}$ used in a deterministic pushdown state–alphabet hierarchy, we obtain a positive decidability result for the full *epsilon-free* class, allowing arbitrary replacement words even on input 0. The argument is not a fixed push-length hypothesis, a zero-pop restriction on the competitors, or a bound for just a family of aligned representations. Its key input is a property of the *language*: every prefix of an accepted source word has an accepted zero-only completion of computably bounded length when the number of input ones is bounded.

Our route is proof-theoretic rather than complexity-theoretic. For each finite number H of ones we prove an exact deletion normalization with a horizon-dependent bound K_H. The resulting

finite consistency problems can be decided by PDA emptiness. The parent map from level $H+1$ to H *deletes* long rules; it never modifies a retained rule. Koenig's lemma therefore produces a coherent sequence of partial tables that stabilizes after finitely many rule-definition events. For the positive case we enumerate finite machines and use the decidable language-equivalence problem for deterministic pushdown automata, established by Sénizergues [5]. We additionally give a finite verification format for infeasible horizons and determine the first infeasible horizon in a small explicit example.

Precise scope. We distinguish epsilon-free competitors from the unrestricted deterministic epsilon-PDA model. Epsilon-enabled machines may decrease the height of their stack without reading a zero; consequently the zero-drain calibration used here no longer bounds their physical stack heights. No general epsilon-elimination theorem with fixed (q, g) is asserted. We make no priority claim for generic compactness techniques and do not claim a solution of the complete two-dimensional Pareto problem. General state/stack-symbol tradeoffs are an established topic in descriptional complexity [6, 7].

Relation to earlier work and contribution. The binary source family $B_{m,k}$ and its representation-independent state–stack product lower bound were developed in the author's earlier binary hierarchy paper [1]. A separate companion study investigates exact Pareto resource corners and uniform frontier width in the *epsilon-enabled* model [2]. The contribution here is different: a decision procedure for the *existence* of equivalent *epsilon-free* recognizers under fixed state and stack-alphabet budgets, without assuming a uniform limit on transition replacement lengths; an effective zero-drain calibration criterion; and finitely checkable negative resource certificates. The paper does not claim that König's lemma, pushdown-language emptiness, or deterministic pushdown equivalence are new. The numerical $B_{3,1}$ example is supported by archived finite computations, with its remaining independent audit dependencies stated explicitly.

Main contributions. Relative to the author's earlier hierarchy theorem [1], which supplies the language family and a state–stack product lower bound, this paper answers a different *synthesis/existence* question: a competing machine is not given, and a priori infinitely many transition tables fit a fixed (q, g) budget because replacement words can be arbitrarily long. The new mathematical component is the *language-driven deletion bound*, which makes the finite-horizon search finite without imposing a normal form on unused transitions; the *coherent deletion compactness* argument then turns feasible horizons into one finite global table. This supports both a decider and a finite-word NO certificate. The calibrated-language extension shows exactly which accepted-prefix property powers the proof. The result is distinct from deciding equivalence between two *given* DPDAs [5, 3], and from decidability questions about push-operation complexity [4]. These classical results and König's lemma are inputs or context, not claimed new contributions.

2 Physical model and the binary source

A deterministic pushdown automaton is a tuple

$$M = (Q, \{0,1\}, \Gamma, \delta, q_0, Z_0, F)$$

with a single physical initial stack symbol $Z_0 \in \Gamma$, $q_0 \in Q$, final states $F \subseteq Q$, and a partial transition function

$$\delta : Q \times \Gamma \times (\{0,1\} \cup \{\varepsilon\}) \rightharpoonup Q \times \Gamma^*.$$

For each (q, X), the presence of an ε-transition excludes every real-input transition. No transition originates with the stack physically empty. A word is accepted exactly when the entire input has been consumed and the machine reaches both a final control state and the *physically empty*

stack. An *epsilon-free* machine has no epsilon transitions at all; its real-input transitions may nevertheless use any finite replacement words and need not pop on 0. We write stack words with the top on the right. An arbitrary finite table always has some finite maximum push length, but no common maximum is specified across competing tables.

For $m \geq 2$, $k \geq 1$, put $\ell = mk + 1$. The canonical machine $U_{m,k}$ has m states $q_1, \ldots, q_m$, $k + 1$ symbols $A, M_1, \ldots, M_k$, initial configuration (q_1, A), and every state final. Let π exchange 1 and 2 and fix $3, \ldots, m$. Enumerate ℓ visible modes (r_j, X_j) by

$$(r_1, X_1) = (q_2, A), \qquad (r_{1+(a-1)k+t}, X_{1+(a-1)k+t}) = (q_{\pi(a)}, M_t)$$

for $1 \leq a \leq m$ and $1 \leq t \leq k$. In particular $r_2 = q_2$. The entire transition table consists of the following specified rules, with any unspecified transition undefined:

$$\begin{aligned}
\delta(q_1, A, 1) &= (q_2, X_\ell X_{\ell-1} \cdots X_1), \\
\delta(q_i, A, 0) &= (q_i, \varepsilon), && 1 \leq i \leq m, \\
\delta(r_j, X_j, 0) &= (r_{j+1}, \varepsilon), && 1 \leq j < \ell, \\
\delta(r_\ell, X_\ell, 0) &= (q_1, \varepsilon), \\
\delta(r_j, X_j, 1) &= (r_j, A^j X_j), && 1 \leq j \leq \ell.
\end{aligned}$$

At (q_2, A) the two apparent zero rules coincide, so the machine is deterministic. Define $B_{m,k} = \mathcal{L}(U_{m,k})$.

Given positive integers (q, g), define $\mathsf{SAT}_{\mathrm{rt}}(m, k, q, g)$ to mean that an epsilon-free deterministic PDA with at most q controls, at most g physical stack symbols and the foregoing single-initial-letter/final-and-empty semantics recognizes $B_{m,k}$ on *every* finite binary input. Transition replacements are unbounded a priori.

3 Finite-ones language calibration

For each $H \geq 0$, let

$$R_H = \{w \in \{0,1\}^* : \#_1(w) \leq H\}, \qquad S_H = B_{m,k} \cap R_H, \qquad K_H = \max(1, \ell H).$$

The set R_H is regular and infinite, whereas the positive language slice S_H will be finite.

Lemma 3.1 (Zero completions of live source prefixes)**.** *For every prefix u of any word accepted by $U_{m,k}$, its unique reached source configuration has some stack height $a(u) \geq 0$ and*

$$u0^{a(u)} \in B_{m,k}, \qquad a(u) \leq K_H \quad \text{if } \#_1(u) \leq H.$$

For $H \geq 1$, every $v \in S_H$ has at most ℓH zeros and therefore $|v| \leq H(\ell+1)$. For $H = 0$, $S_0 = \{0\}$.

Proof. Every defined source input-0 move removes precisely one stack letter, and every source control state is final. Therefore $a(u)$ zeros drain exactly the reached stack, and no shorter or longer pure-zero suffix can be accepted. Initially $a(\varepsilon) = 1$. The initial input 1, if used, changes the height from 1 to ℓ. Every later defined 1-move increases height by some $j \in \{1, \ldots, \ell\}$; a 0-move decreases it by one. Thus for any live prefix containing $h \geq 1$ ones, $a(u) \leq \ell h$. If $h = 0$, the only live prefixes are the empty word and 0, with heights 1 and 0. The terminal stack of an accepted word is empty, so the number of zero moves does not exceed the initial height plus all increases from ones, which is at most ℓH for $H \geq 1$. The no-one language consists only of the singleton word 0. □

Lemma 3.2 (Finite positive slices)**.** *Each S_H is finite and effectively enumerable by testing the finitely many words of length at most $H(\ell + 1)$ for $H \geq 1$ and by $S_0 = \{0\}$ for $H = 0$.*

Proof. Immediate from Lemma 3.1 and effective membership in $U_{m,k}$. □

4 Deletion-only normalization

Given an epsilon-free table M and a positive integer K, let $\pi_K(M)$ be the same machine with the same physical state and alphabet labels, initial site and final set, but *with every real-input transition of replacement-word length greater than K deleted*. No other transition is modified.

Lemma 4.1 (Finite-horizon push normalization)**.** *Suppose an epsilon-free deterministic PDA M agrees with $B_{m,k}$ on R_H. Then $\pi_{K_H}(M)$ still agrees with $B_{m,k}$ on R_H:*

$$\mathcal{L}(\pi_{K_H}(M)) \cap R_H = S_H.$$

This does not require M to pop on input 0*.*

Proof. Removing transitions cannot create an accepting computation. For preservation of positive words, take any $v \in S_H$ and any prefix $u \preceq v$. By Lemma 3.1, the accepted target word $u0^{a(u)} \in S_H$ has $a(u) \le K_H$. Thus M accepts that word, and because it is deterministic and epsilon-free, the configuration of M after u in this computation is exactly the configuration reached after u in its accepting computation on v.

In any epsilon-free PDA, consuming one 0 can decrease the physical stack height by at most one: it replaces one visible symbol with a word of nonnegative length. Accepting the all-zero suffix $0^{a(u)}$ by a physically empty stack therefore implies

$$\mathrm{height}_M(u) \le a(u) \le K_H.$$

Consider a real input transition used on the accepting computation of v, and let u denote the prefix *after* that transition. Its replacement word is a contiguous top block of the new physical stack, so its length is at most $\mathrm{height}_M(u) \le K_H$. None of the transitions of the accepting computation on v is deleted. Therefore all positive words in S_H are preserved, while no new negative word can be accepted. □

Remark 4.2 (Why epsilon-free matters)**.** *With silent transitions, the machine might erase an arbitrarily large stack before consuming the first zero of a correct zero-only completion. Its height at u would then not be bounded by $a(u)$, and the preceding deletion argument would fail. No fixed-resource epsilon-elimination construction is supplied here.*

5 Finite-level decision and compactness

Write $\mathsf{FEAS}_H(q, g)$ if there is an epsilon-free at-most-(q, g) table M such that

$$\mathcal{L}(M) \cap R_H = S_H.$$

We may pad controls and alphabet letters that are unused, so it suffices to consider tables over fixed labeled sets of exactly q controls and g letters.

Lemma 5.1 (Effective finite-horizon consistency)**.** *The Boolean value $\mathsf{FEAS}_H(q, g)$ is decidable for each fixed H, m, k, q, g.*

Proof. By Lemma 4.1, if FEAS_H holds then a witnessing table has no replacement word longer than K_H. There are only finitely many such tables: at each of the $2qg$ input/state/top slots we choose either an undefined transition or one of $q \sum_{j=0}^{K_H} g^j$ possible state/replacement pairs. There are also finitely many choices of final subset and initial labels.

For each candidate, acceptance of every word in the finite set S_H is effectively checked. To exclude all false positives without enumerating the infinite set $R_H \setminus S_H$, construct a DFA for that regular language and decide whether the intersection of its language with $\mathcal{L}(M)$ is empty. Intersection of a PDA language with a regular language is effectively context-free, and emptiness of a context-free language is decidable. Every candidate can thus be tested exactly, and only finitely many candidates exist. □

For $H \geq 0$, denote by T_H the *finite* family of all labeled tables with pushes of length at most K_H and with the correct R_H-behavior. The next statement is the compactness core.

Theorem 5.2 (Deletion compactness)**.** *For all fixed m, k, q, g,*

$$\mathsf{SAT}_{\mathrm{rt}}(m, k, q, g) \quad \Longleftrightarrow \quad \forall H \geq 0\ \mathsf{FEAS}_H(q, g).$$

Proof. The forward implication is immediate. For the converse, assume every T_H is nonempty. If a table $M \in T_{H+1}$, applying the deletion map π_{K_H} yields an element of T_H: indeed M agrees with $B_{m,k}$ on R_{H+1} and hence on R_H, and Lemma 4.1 applies. Therefore the maps

$$\pi_{K_H} : T_{H+1} \longrightarrow T_H$$

are well defined. All T_H are finite. Form the forest whose vertices at level H are tables in T_H, with each vertex at $H+1$ joined to its image at H. Since T_0 is finite, this is a finitely branching forest with vertices at arbitrarily large levels; Koenig's lemma gives an infinite compatible branch

$$M_0, M_1, \ldots, \quad \pi_{K_H}(M_{H+1}) = M_H.$$

Each of the $2qg$ possible real-input transition slots has a simple behavior along this branch. If a transition is defined at level H, its replacement word has length at most K_H and therefore survives every subsequent deletion projection. It must remain *the same transition* at all later levels. Conversely an undefined slot may become defined, but once defined cannot change or be removed. It can make this change at most once. Administrative choices (initial site, final-state set) are preserved by every projection. Thus there exists an index H_* after which the entire labeled table is constant, say $M_H = M_*$ for every $H \geq H_*$. Because M_H agrees with $B_{m,k}$ on R_H, the fixed M_* agrees with the source on *every* R_H. Their union is $\{0,1\}^*$, so $\mathcal{L}(M_*) = B_{m,k}$. □

6 A terminating resource-existence decision procedure

Lemma 6.1 (Effective conversion of the acceptance convention)**.** *Given a deterministic PDA in the present model, one can effectively construct an ordinary final-state accepting deterministic PDA for the same language. This auxiliary conversion need not preserve either resource budget.*

Proof. Insert a fresh permanent bottom marker beneath the original singleton initial symbol. Simulate every original transition above this marker. For each original final control, when the marker becomes exposed, allow a single forced ε-move into a designated accepting sink without input-consuming transitions. If the marker is reached in an original nonfinal control, no such move is provided. No original transition inspects an empty physical store, so no ordinary action competes at the marker, and determinism is preserved. If unread input remains when the accepting sink is entered, that input cannot be consumed and hence the word is not accepted. Thus exactly the original input words ending in a final control with an empty physical store are accepted in the transformed convention. □

Theorem 6.2 (Epsilon-free fixed-resource decidability)**.** *For every $m \geq 2, k \geq 1$ and every $q, g \geq 1$, the statement $\mathsf{SAT}_{\mathrm{rt}}(m, k, q, g)$ is decidable. Competitor zero-transitions may be arbitrary finite replacements, and no common push-length bound is supplied.*

Proof. Run two effective processes in parallel.

1. *Positive semidecision:* enumerate all finite labeled epsilon-free at-most-(q, g) tables in increasing order of maximum replacement length. Use the decidability of deterministic PDA language equivalence [5] to compare each candidate against the explicit canonical source $U_{m,k}$ (using Lemma 6.1 to match the acceptance convention of the equivalence algorithm; this conversion need not preserve resource counts). If an equivalent machine exists, eventually find one and return YES together with its transition table.

2. *Negative semidecision:* for $H = 0, 1, 2, \ldots$ decide $\mathsf{FEAS}_H(q, g)$ using Lemma 5.1. At the first negative level return NO with the finite-level table-exhaustion certificate.

If a correct epsilon-free table exists, the positive process terminates. Otherwise Theorem 5.2 implies that some finite horizon is infeasible, so the negative process terminates. Exactly one of these outcomes is correct. The dovetail thus terminates in all cases. □

Corollary 6.3 (Some computable replacement bound exists)**.** *There is a total computable function $F(m, k, q, g)$ such that whenever an epsilon-free (q, g) recognizer for $B_{m,k}$ exists, one exists with all replacement lengths at most $F(m, k, q, g)$.*

Proof. Run the decision procedure. On a negative instance return 0. On a positive instance, enumerate until a globally equivalent table is found and return its maximum replacement length. The value is total and has the claimed existential property. No useful elementary or complexity estimate is asserted. □

7 Finite refutation packets

The negative part of Theorem 6.2 can be made independently checkable after a finite horizon is found. This is useful because a report that "all tables failed" is not, by itself, a reproducible mathematical certificate.

Definition 7.1 (Horizon-refutation packet)**.** *For fixed integers (m, k, q, g, H), let $\mathcal{C}_H$ be the explicit finite set of* labeled *epsilon-free tables with the specified budgets and replacement words of length at most K_H. A* horizon-refutation packet *is a finite set of binary words $W \subseteq R_H$ such that for each $M \in \mathcal{C}_H$ there exists a word $w \in W$ on which M and $U_{m,k}$ disagree. An equivalent indexed representation stores one witness w_M for every table M.*

Theorem 7.2 (Finite verifiable negative certificates)**.** *For every fixed (m, k, q, g) the following statements are equivalent:*

(i) no epsilon-free machine with the prescribed resources recognizes $B_{m,k}$;

(ii) for some $H \geq 0$ there exists a horizon-refutation packet (H, W).

Given such a packet, its validity can be verified by finite enumeration and ordinary finite-word simulation; a deterministic-PDA equivalence oracle is not *needed for certificate checking. One can choose $|W| \leq |\mathcal{C}_H| \leq qg\, 2^q \big(1 + q \sum_{j=0}^{K_H} g^j\big)^{2qg}$ by assigning at most one counterexample per table.*

Proof. Assume (i). By Theorem 5.2, for some H the finite table family T_H is empty. Thus every table $M \in \mathcal{C}_H$ disagrees with $U_{m,k}$ on at least one word $w_M \in R_H$. Because $\mathcal{C}_H$ is finite, selecting one such word per table produces a finite packet. The witnesses are effective: if a candidate misses a word of finite S_H, use that word; otherwise intersect the candidate language with the regular set $R_H \setminus S_H$, and extract an accepted word from a nonempty PDA–DFA intersection.
Conversely, let (H, W) be a valid packet and suppose a globally equivalent epsilon-free M existed. Since M agrees with the source throughout R_H, Lemma 4.1 would give a bounded table $\pi_{K_H}(M) \in \mathcal{C}_H$ agreeing on R_H, including every word of W. This contradicts the packet. To verify a claimed packet, an independent checker enumerates every table of $\mathcal{C}_H$, recomputes membership of each finitely listed word in the explicit source, and checks that each candidate fails on at least one word. No infinite-input or global-equivalence assertion is needed in this final verification step. The mathematical reduction from arbitrary rewrite lengths still rests on Lemma 4.1; the packet *alone*, divorced from that lemma, must not be mistaken for an exhaustion of unbounded tables. □

Remark 7.3 (Finite negative witnesses versus a cutoff)**.** *The packet may contain relatively long words or many separate counterexamples; no efficient size or complexity upper bound is claimed. It is a proof-carrying* negative *output. A successful bounded table for one finite horizon is* not *a global positive certificate: the positive output is a genuinely equivalent finite machine, established using deterministic-PDA equivalence.*

Corollary 7.4 (Constructive yes/no outcomes)**.** *For the family $B_{m,k}$ and fixed q, g, the resource-existence decision can be organized to return either (i) a concrete equivalent epsilon-free transition table, or (ii) a finite horizon H and a finite input-word packet $W \subseteq R_H$ refuting every candidate transition table bounded by K_H. The correctness of a* NO *output is checkable using only finite-word simulation and the normalization theorem; checking a* YES *output uses deterministic-PDA equivalence.*

Proof. On a positive instance, enumerate candidate tables and stop at the first one certified equivalent by the deterministic-PDA equivalence procedure. On a negative instance, compactness gives a first infeasible horizon; the construction in Theorem 7.2 extracts one finite counterexample for every bounded table and packages them as W. □

8 A calibrated zero-drain principle

The proof can be separated from the exact grammar of $B_{m,k}$.

Definition 8.1 (Effective zero-drain calibration)**.** *Let $T \subseteq \{0,1\}^*$ be a language given by an effective deterministic pushdown automaton under the same physical empty-store/final-state acceptance convention. A* zero-drain calibration *is an algorithm which on input H computes a positive integer K_H such that $(K_H)_{H\geq 0}$ is nondecreasing and has the following property: for every $H \geq 0$, every accepted word $v \in T \cap R_H$, and every prefix $u \preceq v$, there is some $a \in \{0, \ldots, K_H\}$ with $u0^a \in T$.*

Lemma 8.2 (Drain calibration already implies finite sections)**.** *Every effectively zero-drain calibrated language T in this model has finite, effectively enumerable sections $T \cap R_H$. In fact, every $v \in T \cap R_H$ satisfies*

$$|v| \leq H + (H+1)K_H.$$

Proof. Every language accepted by this physically-empty-stack/no-empty-move model is prefix-free: if an input word has drained the store and been accepted, no proper extension can also be accepted. Decompose an accepted word v with $h \leq H$ ones into its $h+1$ maximal blocks of zeros separated by ones. Immediately before any zero block, at a prefix $u \preceq v$, there is by calibration an accepted completion $u0^a$ with $a \leq K_H$. If the block contains more than a zeros, then $u0^a$ is a proper accepted prefix of v, impossible. Thus every zero block has length at most K_H and $|v| \leq h + (h+1)K_H \leq H + (H+1)K_H$. Testing the finitely many words of that length against the given target DPDA enumerates $T \cap R_H$. □

Theorem 8.3 (General calibrated resource decision)**.** *Given a deterministic PDA for T with the acceptance convention of Section 2,* together with an algorithm computing a valid zero-drain calibration (K_H)*, and any $q, g \geq 1$, one can decide whether T has an equivalent epsilon-free recognizer with at most q states and g stack letters, with arbitrary finite replacement words. In particular, for every fixed effectively calibrated T, this resource-existence predicate is decidable.*

Proof. In Lemma 4.1 use the accepted drain $u0^a \in T$ supplied by calibration in place of the canonical source height $a(u)$. The height inequality for every epsilon-free competitor remains $\mathrm{height}_M(u) \leq a \leq K_H$. Truncation, finiteness of the candidate tables, the finite positive set from Lemma 8.2, and regular-language emptiness for all false positives give decidable H-consistency. The deletion-only compactness proof is unchanged. Finally, the given target is a deterministic

PDA, so the positive enumeration can use deterministic PDA equivalence. Dovetail the two semidecision procedures as before. □

Remark 8.4 (Status and limits of the general criterion)**.** *Theorem 8.3 is a logical abstraction of the proof for $B_{m,k}$, not a claim of priority or of a classification of all languages admitting such calibrations. Its hypotheses impose a strong* accepted-prefix *zero-completion property, not simply a short zero-completion from a few chosen source rays. The calibration algorithm is part of the* effective promise*: the theorem does not assert an algorithm detecting this property from an arbitrary given target DPDA without a supplied calibration. There is no claim about epsilon-enabled competitors. Sénizergues' theorem is used for the positive existence branch, while all finite negative-horizon checks require only ordinary PDA emptiness in the calibrated setting.*

9 A transparent finite infeasibility certificate

An explicit small example illustrates why a negative H-test can be decisive without being a global machine enumeration.

Proposition 9.1. *No epsilon-free deterministic PDA with one state and one physical stack letter recognizes $B_{2,1}$. More precisely, $H = 2$ is its* first infeasible ones-horizon*: $\mathsf{FEAS}_0(1,1) = \mathsf{FEAS}_1(1,1) = \mathsf{true}$ and $\mathsf{FEAS}_2(1,1) = \mathsf{false}$.*

Proof. Here $\ell = 3$. By direct evaluation of $U_{2,1}$,

$$0 \in B_{2,1}, \qquad 1000 \in B_{2,1}, \qquad 110000 \in B_{2,1}.$$

Any epsilon-free one-control/one-letter table agreeing with these words must mark its sole control final. Accepting 0 forces its input-0 transition to replace the single symbol by ε. Accepting 1000 then forces the input-1 transition to replace the single symbol by three copies of itself. On 110000 the two ones leave five stack copies (one initial letter, plus two increments of two); four zeros remove only four copies, so this word is rejected. All three displayed positive words belong to R_2, contradicting R_2-agreement. The argument is independent of any finite push cutoff beyond those forced transitions. Indeed the three-word set $W = \{0, 1000, 110000\}$ is an $H = 2$ horizon-refutation packet: all three words are source-positive, and they jointly contradict every one-state/one-letter transition table. Under $K_2 = 6$, exactly $2(1+7)^2 = 128$ labeled tables are present (two possible final-state sets and eight choices for each real-input rule, including undefined); exhaustive simulation confirms that none accepts all three words.

For comparison, at $H = 1$ the one-state/one-letter epsilon-free machine with its sole state final, $X \xrightarrow{0} \varepsilon$ and $X \xrightarrow{1} X^3$, recognizes precisely the two source words 0 and 1000 among all words in R_1. Indeed an accepted word can contain no ones or exactly one one; the latter must be followed by exactly three zeros. Therefore $\mathsf{FEAS}_1(1,1)$ holds, and the same table works at $H = 0$. This proves the claimed minimality of the obstructing horizon. □

The proposition is deliberately pedagogical: it is not a new optimal resource threshold and does not establish the general $g = 1$ impossibility for all control budgets.

10 Discussion and conclusion

The principal achievement is the fixed-resource *existence decision* for the entire binary hierarchy $B_{m,k}$ in the epsilon-free model, rather than an equivalence test for two specified machines. A language-derived zero-completion bound, deletion-only projection and stabilization of finitely many transition slots are what overcome the unbounded-replacement obstacle. The separate negative-certificate theorem makes resource nonexistence witnessable by finitely many words; the

generalized calibration theorem identifies a larger, explicitly promised class of target languages to which the mechanism applies. No practicable complexity or small universal bound on the first infeasible horizon is claimed.

The additional computational $B_{3,1}$ example is recorded in the appendix with its hypotheses and verification boundary, not as a dependency of the preceding theorems. The epsilon-enabled fixed-resource existence question, as well as a closed-form complete state–stack Pareto classification of all $B_{m,k}$, remains open in the scope of this work.

A Computational Pareto case study for $B_{3,1}$

The abstract finite-ones decision procedure does not by itself identify a practically small obstruction horizon. A separate archived line of structural reductions and executable certificates (G72–G89) supplies a more demanding example. Its mathematical scope is the *epsilon-free* resource class, not arbitrary epsilon-enabled DPDAs. We now isolate three key necessary steps, including their exact logical and computational dependencies.

A.1 Root separation without a normal-form assumption

In the next two lemmas stack tops are written on the left. Put $\ell = mk + 1$, and $P_j = 10^j$ for $0 \le j < \ell$. The canonical $B_{m,k}$ source has the exact completion profiles

$$P_j 1^n 0^a \in B_{m,k} \quad \Longleftrightarrow \quad a = \ell - j + (j+1)n \qquad (n, a \ge 0).$$

The statement concerns actual physical control/top pairs of any competing machine; it does not assume source-aligned physical stack letters.

Lemma A.1 (G72: physical root exclusion)**.** *In every equivalent epsilon-free deterministic pushdown automaton in the stated one-initial-letter final-and-empty model, the initial physical control/top site is different from the physical site exposed immediately after each P_j, $0 \le j < \ell$. The target may have any finite number of controls or stack symbols and arbitrary finite* 0- *and* 1*-rewrites.*

Proof. Write the initial configuration as (p, A), and suppose a clean-prefix configuration exposes the same site (p, A) above a fixed buried word β. For $j \ge 1$, consider the singleton-root computations on $1^n 0^{\ell+n-1}$, $n \ge 1$. Just after the 1^n part, their stacks are nonempty, or the subsequent zero suffix could never be consumed. Therefore, by deterministic top-only locality, starting instead at $(p, A\beta)$ performs the identical upper-layer 1^n computation without touching β. The subsequent $\ell + n - 1$ zero moves erase this common upper layer and enter some control f_n, which is final in the singleton-root computation. For the clean-prefix completion there must then be $\ell - j + (j+1)n - (\ell + n - 1) = j(n-1) + 1$ further zero moves emptying the *fixed* buried word β from control f_n. At a fixed control and a fixed physical stack there is at most one accepted pure-zero length. Since f_n ranges over finitely many controls, the possible lengths are bounded, contradicting $j(n-1) + 1 \to \infty$.

For $j = 0$, assume instead that the machine again exposes (p, A) after the initial input 1. Its first 1 rule must write $A\beta$ and return to p. The word 0 is accepted from (p, A), so the 0 rule pops A and enters some final state s. Here $\beta \ne \varepsilon$, because otherwise the configurations before and after reading 1 would coincide, whereas their unique accepted zero-completion lengths are 1 and ℓ. The physical configurations after 10 and 110 therefore have the same control s and stacks β and $\beta\beta$, respectively. The genuine source accepts the two words

$$10\,11\,0^{\ell+3} \qquad \text{and} \qquad 110\,11\,0^{\ell+2}.$$

From the first configuration, the physical 11 computation must retain a nonempty upper stack word W and then erase W in exactly $\ell + 3$ zeros. The same 11 steps from $(s, \beta\beta)$ cannot touch the

buried second copy of β and therefore leave $W\beta$ in the same control. Its zero-only computation cannot erase the common W upper layer in merely $\ell+2$ zeros. This contradicts the second genuine acceptance, completing the root exclusion. □

A.2 Two controls and the first shared-source input

Lemma A.2 (G74: immediate POP at a shared clean site)**.** *Suppose an epsilon-free two-control DPDA correctly recognizes $B_{m,k}$. If two distinct clean prefixes P_i, P_j expose the same physical control/top site, then the input-*1 *rewrite at that site is the empty stack word. No global* 0*-POP hypothesis is required.*

Proof. The configurations have the form $(p, A\beta)$ and $(p, A\gamma)$. Suppose the site's first input-1 rule writes a nonempty physical word V. At counts $n = 0, 1, 2$ of further input ones, the upper-layer computations from the two configurations remain identical: at $n = 0$ the common upper word is A, at $n = 1$ it is V, and at $n = 2$ the second 1 rewrites the top of V in exactly the same way on both sides. The resulting upper word at $n = 2$ may be empty; that does not affect this argument. Denote the common upper word by W_n and the common control by s_n. An accepted all-zero suffix must remove W_n before inspecting either buried tail. The first-return zero cost and exit control e_n of this upper erasure are identical for both runs. Subtracting the unique accepted zero-completion lengths leaves

$$\tau_{e_n}(\beta) - \tau_{e_n}(\gamma) = \big[\ell - i + (i+1)n\big] - \big[\ell - j + (j+1)n\big] = (j-i)(1-n), \quad n = 0, 1, 2,$$

where $\tau_e(\xi)$ is the successful zero-emptying length of fixed physical tail ξ from control e. Each fixed e determines at most one such difference. Since the machine has only two controls, at most two distinct differences can occur, whereas the three displayed values are distinct because $i \neq j$. This contradiction forces the first 1 rewrite to be a physical POP. □

A.3 G71/G76: complete finite clean-zero interfaces

The following enumeration accounts for the precise class entering the computational certificates; it makes no claim that the enumerated partial tables recognize the whole language.

Lemma A.3 (Complete clean-zero skeleton bound)**.** *Normalize the unique initial physical site of a putative epsilon-free $(q, g) = (2, 3)$ recognizer of $B_{3,1}$ to $(0, 0)$. Every accepting computation on* 0 *and* 10000 *is represented by one of finitely many* clean-zero skeletons*, including non-POP real-*0 *rules and either possible cardinality of the final-state set. The four zero-step replacement lengths have precisely* 14 *possible ordered-forest shapes, partitioned by the first input-*1 *header length $d = 1, 2, 3, 4$ into $5, 5, 3, 1$ shapes.*

Proof. Acceptance of 0 forces the root's real-0 rule to POP its only physical symbol and to enter a final control. Let the first real-1 write $d \geq 1$ symbols, and let U_j be the word written by the j-th real-0 rule along the accepting run on 10000, $0 \leq j \leq 3$. With $h_0 = d$ and $h_{j+1} = h_j - 1 + |U_j|$, each of the first four heights is positive and $h_4 = 0$. Therefore $d \leq 4$ and

$$\sum_{j=0}^{3} |U_j| = 4 - d, \qquad |U_3| = 0, \qquad 1 \leq h_j \leq 4 - j.$$

Every such length vector is exactly the preorder outdegree sequence of a plane ordered forest with four nodes. Adjoining a single auxiliary root gives one of $C_4 = 14$ plane trees. Conversely the fourteen legal shape vectors satisfy the necessary positive-height and final-empty conditions. For any chosen vector, all two-control destinations and three-letter replacement words can be enumerated, assigning a transition only when its physical site is first used; a reused site must keep the same rule. This tests every deterministic partial table on the required clean scan, without constraining any unvisited site. □

An independent *forest-first* enumerator, which fixes one of these fourteen length vectors before choosing any state/letter labels, reproduced 9,107 one-final and 15,885 both-final raw clean skeletons, for 24,992 in total. The repaired G71 root/singleton-source filters leave $3{,}808 + 10{,}332 = 14{,}140$ necessary raw interfaces. Replacing the weaker G71 two-final root filter by Lemma A.1 leaves 11,424 raw interfaces; quotienting only by the swap of the two noninitial physical stack-letter names gives 5,712 orbits. Exactly 5,040 orbits have four distinct clean source sites, disjoint from the root; the other 672 have a shared source site. Each of the fourteen clean-zero forest shapes contributes *exactly 360* of those 5,040 source-distinct orbits. These are exact finite enumeration counts, not an assumption that the input-0 transition always POPs.

A.4 G75: sound pruning of source-sharing interfaces

If a physical singleton site accepts $1^n 0^{a+bn}$ for all sufficiently large n, then the same exposed site with a fixed buried tail cannot have an accepted eventually affine zero-completion profile with a different slope: the singleton upper run shields the tail, and after its upper zero-drain the possible tail completion lengths belong to a finite set indexed by the finite control states. For $B_{3,1}$ the physical root is a singleton of slope one, whereas the physical clean site at 1000 is a singleton of slope four.

Lemma A.2 first forces an immediate real-input-1 POP whenever two of P_0, P_1, P_2 share a clean physical top site. The resulting lower tails must each accept a five-zero continuation from one common control. When a currently undefined physical input-0 rule is first used with t zeros left and a buried tail of length e, any replacement U on an accepting computation must satisfy

$$|U| + e \le t - 1.$$

Indeed only $t - 1$ real zero moves remain after that transition, and one real move reduces physical height by at most one. Thus the finite backtracking of new zero rules along *these accepted runs* is complete even with arbitrary non-POP rules off the runs. Together with the singleton-slope constraints and the authentic D-debt one-dead prefixes, this yields the necessary-orbit counts

$$672 \xrightarrow{\text{G74}} 126 \xrightarrow{\text{G75}} 110, \qquad 110 = 36 \text{ (one-final)} + 74 \text{ (both-final)}.$$

The 110 are a finite *superset of necessary interfaces*, not a list of complete correct machines. Their remaining physical zero/one rules are subsequently handled by the G89 computation.

A.5 G78: soundness of the zero-nonproductive exception gate

The 5,040 source-distinct clean interfaces occupy five of the six physical state/top-letter sites. Write E for the sixth. If E has no finite zero first-return, it cannot occur on any accepting run of a genuinely correct recognizer: every live source prefix admits a pure-zero accepting completion, which would necessarily remove its currently exposed top-origin layer. Hence *both* real input rules at E may be deleted without affecting any successful computation. This is an exact normalization for this branch, not a general zero-POP normal form.

For each of the four clean sites Z_j the real input-1 rule is required by the accepted word $P_j 10^5$. If its current physical stack has height h_j, its replacement word V_j must satisfy

$$|V_j| + h_j - 1 \le 5 \quad \Longrightarrow \quad |V_j| \le 6 - h_j \le 5.$$

Thus every source-1 rule needed in the finite contradiction has a mathematically proved finite search domain. The remaining root-1 rule and the five productive zero rules were fixed by the accepted clean scan. The archived G78 certificate imposes the authentic seventeen three-1 accepted completions and the three D-debt one-dead tests. Every surviving candidate must be supported in each single-source domain and each cross-source necessary binary constraint; arc consistency can

only *delete* a choice when it lacks such support. The four nonempty arc-consistent interfaces then admit exactly ten complete source-rule tuples. Direct physical simulation excludes two by those same tests; all eight others reject the authentic accepted word 10101100000000. The independent reproduction found precisely

$$5{,}040 \longrightarrow 4 \longrightarrow 10 \longrightarrow 8 \longrightarrow 0.$$

The code's direct last-stage simulation is essential: successful local constraint propagation alone is not claimed to prove UNSAT. The normalization, upper bounds, and monotonicity arguments establish why G78 tests a necessary superset of *all* zero-nonproductive-E competitors. The numerical exhaustion remains a computational premise and depends on archived finite-search code that is not part of this PDF-only submission. The numerical exhaustion is therefore not independently executable from the submitted PDF alone.

A.6 G79: where the numerical cutoff uses computation

For $B_{3,1}$ and resource budget $(2, 3)$, G72 and the clean-prefix singleton and transition-count constraints yield a finite necessary set of source interfaces. In the source-distinct branch, five distinct physical sites are the initial root and the four clean sites; denote the sixth physical site by E. The archived G78 algorithm supplies the following *computational premise*: no such candidate in which both input transitions at E are deleted can simultaneously accept the specified 25 positive source words and reject the three prescribed zero-continued D-debt negative families. This premise was reproduced by executing its original certificate; it is not a consequence of Lemma A.1 or Lemma A.2 alone.

Lemma A.4 (G79: productive exceptional-site bound)**.** *Conditional on the soundness and exhaustive necessary-interface coverage of that G78 computational premise, any globally correct source-distinct* $(2, 3)$ *epsilon-free recognizer of* $B_{3,1}$ *has a finite zero first-return cost* $1 \le c_E \le 9$ *at the exceptional site* E*. Its physical input-*0 *replacement word has length at most* 8*.*

Proof. If a globally correct candidate never exposed E at any prefix of the 25 accepted test words, deleting both E transitions would preserve all 25 accepted runs. It would also preserve every required rejection, since removing transitions cannot introduce acceptance in a deterministic machine. This would contradict the G78 finite premise. Hence some live test prefix w exposes E. An independent literal canonical-source calculation shows that each such prefix has an accepted pure-zero completion of length $h(w) \le 9$; the maximum nine occurs at the prefix 100011. From the competitor's physical configuration after w, those $h(w)$ zero transitions must empty its full stack. In particular, they must erase the currently exposed E-origin layer, so its context-independent first zero-return cost satisfies $1 \le c_E \le 9$. If the actual real input-0 rule at E writes a physical word U_E, its inserted layer needs at least $|U_E|$ further zero moves to be erased, hence $1 + |U_E| \le c_E \le 9$. □

Remark A.5 (Do not confuse a finite-corpus cutoff with normal form)**.** *A physical input-*1 *rule at* E *with a replacement longer than nine cannot occur in any of the above accepted test computations. For the* finite *necessary G78 tests only, such a rule may be omitted. This does not impose a global nine-letter bound on arbitrary correct machines' input-*1 *rules; it is also not an epsilon-elimination theorem. The numerical G79 cutoff is downstream of G78, whereas G72 and G74 are standalone structural statements independent of that computation.*

A.7 G83–G88: an exhaustive exceptional-return cost partition

The certificate-based exclusion of the source-distinct branch is not a single unrestricted finite-table enumeration. It is a disjoint and exhaustive partition by final-state cardinality and by the *first zero-return cost* of the sixth physical site E. We spell out the resource-model arguments that justify that partition.

Lemma A.6 (Physical completeness of short exceptional returns)**.** *In the source-distinct, one-final* $(2,3)$ *epsilon-free class, suppose* E *has a finite zero first-return cost* $c_E \in \{1,2,3,4,5\}$*. For each fixed clean zero table, its physical input-*0 *rule writes a word* V *of length at most* $c_E - 1$*. Moreover, a zero computation exposing the original buried tail cannot visit* E *again before doing so. Consequently, using the already fixed five ordinary zero sites, one can enumerate every physical rule for* $E0$ *by examining finite words* V *with the genuine ordinary-site zero-drain cost* $c_E - 1$*. Across all* 1,680 *alphabet-normalized one-final, source-distinct clean interfaces, the numbers of distinct* (interface, $E0$ rule) *pairs are*

c_E	1	2	3	4	5	total
#	3360	5880	12048	26700	60372	108360.

Proof. One real input-0 transition removes its original top symbol and writes V in its place. Each of the $|V|$ inserted symbols must later be removed, with at most one removed per zero, before the older tail is exposed. Hence $c_E \geq 1 + |V|$. If the first-return computation exposed the site E again while the original E-origin layer was still above its tail, determinism would repeat the same $E0$ action on the new upper layer, never reach the older tail, and contradict finite first return. Therefore after the first $E0$ step only the five fixed ordinary zero sites are visited until the inserted word drains. The preceding upper bound limits V to lengths at most four. Enumerating its two possible target controls and all such words, computing their exact zero drain through the fixed ordinary rules, and retaining exactly those of cost $c_E - 1$ is exhaustive. A second implementation built the clean interfaces by enumerating the fourteen preorder plane-forest shapes *before* assigning control and stack-letter labels; its physical first-return simulation reproduced all displayed counts, independently of the G84–G87 E-rule search implementations. The finite enumeration establishes the counts; the preceding locality argument establishes their completeness. □

Lemma A.7 (G84: reduction for two final controls)**.** *Assume the source-distinct epsilon-free recognizer has both control states final and every physical top site has finite zero first return. Then each of the three authentic zero-live but one-dead D-debt prefixes* 10010*,* 100100*, and* 1001000 *exposes the sole exceptional site* E*, whose input-*1 *rule is undefined. Necessarily its input-*0 *rule is a physical POP returning to the* E *control state; thus* $c_E = 1$*.*

Proof. With all zero returns finite and both states final, any defined input-1 transition from a nonempty stack is followed by some accepting pure-zero continuation, possibly of length zero. Such a transition is therefore forbidden at the three D-debt sites. The initial root and all four clean sites have mandatory input-1 continuations, so each D-debt site must be E. The first two D-debt configurations are separated by one consumed zero yet expose the same physical E site. If the $E0$ replacement were nonempty, its successor would again expose the identical site and could never complete a finite zero return. Thus the replacement is empty, and the successor control must equal the control component of E, as required by the second exposed D-debt site. This proves the claim. □

Lemma A.8 (G83: safe high-cost relaxation)**.** *For* $6 \leq c_E \leq 9$*, every input-*1 *rule at a clean source* P_j *can be selected within the physically necessary length bound* $|V_j| \leq 6 - h_j \leq 5$*, using only the five already known zero sites for its mandatory* $P_j 10^5$ *completion. At all required positive three-one test positions, the third input-*1 *transition also uses an ordinary site, never* E*. Thus replacing the physical* $E0$ *rule by an ideal first-return macro of cost* c_E *and either exit control, and leaving* $E1$ *unconstrained, is a* necessary overapproximation *of all correct competitors in this cost range.*

Proof. A zero computation with at most five steps cannot return from a currently exposed E layer whose first return costs at least six. The clean $P_j 10^5$ computations therefore use no E on their mandatory successful returns. Likewise, immediately before the third input 1 in any of the genuine accepted tests $P_j 10^z 10^v$, the intermediate stack is zero-completable in $5 - z \leq 5$ zeros, so

its exposed site is ordinary. The ideal macro retains every actual first-zero-return cost and exit control while dropping restrictions from the internal $E0$ computation. Hence no correct physical machine is excluded by that relaxation. □

The finite-search checks apply the stated mathematical reductions to labelled partial transition tables. At each stage the admissible physical control/top pairs and source-return conditions determine a finite rule domain; inconsistent partial assignments are discarded, and sound local constraints are propagated before any remaining full tables are evaluated on specified positive and negative source words. Where a search omits a transition, the text explicitly identifies the necessary overapproximation that makes the omission safe. The resulting counts describe the internally executed enumeration, not a substitute for justifying that every globally correct machine enters the enumerated classes. The following is a *complete case partition*, not a claim that every certificate has been freshly re-executed in this manuscript preparation session:

Exceptional zero mode	Final set	Exclusion source
Zero-nonproductive	one or two	G78 finite corpus
$c_E = 6, 7, 8, 9$	one or two	G83 ideal-macro UNSAT
$c_E = 1, 2, 3, 4, 5$	both final	G84 symbolic $c_E = 1$ reduction + finite UNSAT
$c_E = 5$	one final	G85 physical E0 refinement
$c_E = 2, 3$	one final	G86 physical E0 and bounded necessary E1
$c_E = 4$	one final	G87 E1-free overapproximation
$c_E = 1$	one final	G88 full last case

The upper cutoff $c_E \leq 9$ is Lemma A.4; its computational G78 dependency remains explicit. G83's certificate examines all 5,040 clean source-distinct interfaces with both choices for the ideal E-exit control at each high cost; G84 covers all 3,360 both-final clean interfaces and directly rejects all 163 locally compatible full source-rule combinations. For the one-final physical E0 cases, G85 reduces 1,372 refined cost-five combinations to four D-consistent candidates, which all reject a required true word. G86 reports zero cost-two survivors; its unique cost-three relaxed survivor is discharged by bounded actual E1 choices and explicit positive/negative counterexamples. G87 independently reports 26,700 cost-four E0 pairs and no survivors after necessary unary and binary checks. G88 handles cost one.

Current audit distinction. The original G85 and G87 complete certificates were re-executed with successful expected assertions; G85 additionally has a four-trace check proving its last contradiction independent of an unassigned exceptional input-1 action. The G83 costs-six/seven and G84 all-final certificates also passed earlier reruns. In a further 25 October 2026 audit, the entire archived G86 cost-two/cost-three program was re-executed successfully on its original source: it checked all 1,680 normalized one-final clean layouts, 5,880 physical cost-two $E0$ pairs and 12,048 cost-three pairs. The cost-two side had zero survivors. The cost-three side had a single necessary interface with two known-one rule tuples; one misaccepts 10010001000 regardless of the exceptional one-rule. The other tuple required consideration of exactly 242 bounded exceptional-one replacements, 45 of which accepted the first mandatory positive word and seven of which passed the stated 25 positive and D-debt necessary tests. Of these final seven, six reject the genuine positive word 11100100000 and one accepts the genuine negative word 1001000110001. The exact physical tables of those seven final options were exported from a separately instrumented replay and checked by an independent right-top-stack simulator against a separate canonical $B_{3,1}$ interpreter. A subsequent independent four-part G86 constraint solver also reproduced the *entire* cost-two/cost-three physical $E0$ enumeration: $5{,}880 = 1{,}936 + 3{,}944$ cost-two pairs all fail; $12{,}048 = 3{,}908 + 8{,}139 + 1$ cost-three pairs leave just one necessary interface and its two source-one assignments. Those physical assignments coincide with the archived tables up to renaming stack letters. This new solver covers the finite constraint enumeration, whereas the seven-witness interpreter independently checks its terminal rejections. The earlier G101 full-chain

audit remains historical, not a fresh clean-room mathematical proof. A forest-first $E0$ counter also independently confirms all five physical return-cost-domain sizes. These computational agreements do not certify the mathematical necessity of every upstream interface reduction or constitute a proof-assistant verification of the complete case split.

A.8 Referee audit of source-one projection and exceptional-rule independence

The exceptional physical site E is the single state/top-letter site outside the root and four pairwise-distinct clean sites. We record why its possibly very long input-1 rule does *not* invalidate the negative finite-domain checks in the cost-four and cost-five cases.

Lemma A.9 (Safe projection of an unassigned exceptional input rule)**.** *Fix a clean-distinct, one-final, epsilon-free candidate with all six real-*0 *rules having finite zero-first-return. At a clean prefix $P_j = 10^j$ suppose the physical stack has height h_j and the input-*1 *rule at its exposed clean site writes a word V_j. Every globally correct candidate satisfies*

$$|V_j| \le 6 - h_j \le 5.$$

*Consider a test of the prescribed continuation $P_j 1\, 0^z 1\, 0^v$, or its D-debt forbidden variant. Simulate the intermediate z zeros using the actual six input-*0 *rules. If the pending input-*1 *site is E,* discard this particular test constraint*; otherwise impose only its genuine positive/negative zero-completion obligation at the root or one of the four clean sites. All such retained constraints are necessary for a globally correct machine, independently of the value, definedness or output length of its E input-*1 *transition. Deleting source-rule options by repeated binary arc consistency (only when no partner option supports a retained obligation) cannot eliminate a globally correct machine.*

Proof. The source accepts $P_j 1\, 0^5$. The physical height immediately after $P_j 1$ is $|V_j| + h_j - 1$, and each of the following five input-0 steps can decrease height by at most one. Physical empty-store acceptance therefore forces the bound. The real input-0 trace before the pending input-1 is determined by the fixed zero table. If its top site is E, omitting the condition is an *existential relaxation*: every choice of $E1$ is admitted, not just short writes or undefined rules. If it is another known site, the deterministic input-1 action is the root rule, the current clean rule or some other selected clean rule; zero-only completion from the resulting stack is fully determined by the six finite-return zero rules. Each retained positive or negative constraint is consequently necessary. A valid complete machine's four selected clean actions give a simultaneously consistent witness for every binary dependency; a rule can never lack a supporting partner in the domains containing that complete tuple. Induction through each monotone deletion round proves the claim. □

At exceptional cost $c_E = 5$, the initial relaxation requires a little more care. A clean first-1 output that reaches *singleton* E can use the five-zero return exactly, and all later mixed-1 tests of that source are deliberately bypassed at this stage. For any other valid first-1 output, the next accepted five-zero drain cannot expose E: its return would already consume five zeros, leaving no room for a prior zero step or a buried tail. Thus the subsequent constraints only read the five known one-sites, and the actual $E0$ rule is filled in and tested separately. This is a stronger relaxation than deciding or globally bounding $E1$.

Proposition A.10 (An $E1$-independent final G85 witness)**.** *For the four physical candidates surviving G85's cost-five, one-final and source-distinct D-debt tests, the fully determined physical computation after input $u = 100100001$ is in control* 1 *with an* empty physical stack*. No input-*1 *transition at E is read along any of these four computations. On the other hand,*

$$u0^5 = 10010000100000 \in B_{3,1}.$$

Consequently none of these four candidates can be repaired by assigning an arbitrary $E1$ transition.

Finite trace certificate. The four traces were replayed with the six literal physical real-0 rules and the five fixed root/clean real-1 rules. Their final configurations are all $(1, \varepsilon)$; the asserted absence of any $E1$ visit is checked at each input-1 step. An independent literal source simulation verifies the required positive completion. Since no operation is available on the physically empty stack, the five remaining zeros cannot be consumed. The exact recorded trace and its executable assertions are bundled with the v0.7 audit. □

For $c_E = 4$, the literal G87 executable was independently rerun to completion: among its 26,700 possible physical $E0$ rules over all 1,680 normalized one-final clean layouts, 7,998 leave an empty unary clean-rule domain and the remaining 18,702 lose support under monotone binary arc consistency. No option survives even when every test whose pending input-1 site is E is omitted. This proves UNSAT of this *necessary overapproximation*, not an assumed normal form for $E1$. For G86 at exceptional costs two and three, the original source-level projection argument was inspected and the complete program rerun on 25 October 2026. An independently written four-part constraint solver reconstructed all 5,880 cost-two and 12,048 cost-three physical $E0$ alternatives, including the unique surviving relaxed interface at cost three and its two source-one assignments. A separate simulator distinguished the last seven $E1$ options. The finite enumeration is independently reproduced, but the mathematical necessity of the source-interface reduction remains a distinct proof-to-code obligation.

Proposition A.11 (Computational lower certificate)**.** *Assuming the remaining necessary-interface reductions G75 and the computationally exhaustive exceptional-site case split G78–G87, with G72 and G74 justified independently in Lemmas A.1 and A.2, the G88/G89 exhaustive certificates exclude every epsilon-free DPDA for $B_{3,1}$ with at most two controls and three physical stack symbols.*

Certificate decomposition and audit scope. Normalize the initial physical site to $(0, 0)$ and interchange the two noninitial stack-letter labels as needed. Let $P_j = 10^j$ for $j = 0, 1, 2, 3$ and write Z_j for the physical control/top-symbol site after P_j. The archived necessary interface lemmas exclude equality between the initial site and any Z_j, exclude sharing by Z_3, and prove that when two other Z_j share a site their input-1 transition must pop immediately. This partitions all candidate machines into pairwise-distinct and sharing-source interfaces.

For the source-distinct class G78–G87 exclude all exceptional-site zero-return patterns except the final $c_E = 1$ one-final case; the only necessary global finite-cost cutoff $c_E \leq 9$ is justified by Lemma A.4, explicitly conditional on G78. G88 exhausts 1,680 initial interfaces and tests 188,089 physically realizable remaining $E1$-transition choices; of the candidates meeting earlier positive obligations, the final genuine positive word 101100100000000 rejects the remaining 32 cases. For the sharing class, G75 leaves 110 necessary interfaces. G89 enumerates 866 one-final and 6,981 both-final complete physical zero-rule tables. The former are reduced to 18 source-one assignments, each of which incorrectly accepts the genuine source-negative word 10010100000000; the latter all violate a D-debt one-dead constraint. Thus neither side admits a globally correct candidate.

The G72, G74, G78, and G79 scripts as well as the G88/G89 terminal scripts were rerun from archival copies, with successful completion and matching assertions. Independent follow-up solvers reproduced the entire G86 finite constraint scan, the G88 count of 385 compatible source/$E1$ summaries and 188,089 physical $E1$ options (including the last 32 tables), and the G89 count of $7{,}847 = 866 + 6{,}981$ complete zero-rule tables. The 32 final G88 rejects and 18 final G89 erroneous acceptances were checked by separately implemented physical simulators. In the independent G89 audit, an overstrong D-debt filter inherited from an earlier auxiliary check was corrected; the original G89 program was not affected. An independent canonical-source interpreter also checked the 25 positive corpus words, 165 distinct live test prefixes, and the unique prefix of semantic height nine. The displayed computations are exhaustive *within the necessary reductions*; the mathematics of those reductions is a separate proof obligation. In particular, a successful script execution alone is not a formal machine-checked proof of G72–G87. The finite-search computations

and their dependency records are preserved in unpublished working records and are not submitted as code, datasets, or a technical supplement. The numerical case study consequently cannot be independently re-executed from the submitted article alone. The reductions, case partition, test-word obligations, survivor counts, and limits of their interpretation are stated here, but external verification of exhaustive coverage remains a separate obligation. □

Proposition A.12 (Explicit epsilon-free two-control, four-letter source)**.** *There is a deterministic epsilon-free automaton recognizing $B_{3,1}$ with exactly two controls and four physical stack letters, one physical initial letter, and final-and-empty acceptance.*

Proof. Give a complete positive table directly. Take $Q = \{0, 1\}$, $\Gamma = \{a, b, c, d\}$, initial site $(0, b)$, both controls final, and top written on the *left*. All unlisted transitions are undefined:

site	input	(next state, replacement)	site	input	(next state, replacement)
$(0, a)$	0	$(0, \varepsilon)$	$(0, b)$	0	$(0, \varepsilon)$
$(0, b)$	1	$(1, accd)$	$(0, c)$	0	$(0, \varepsilon)$
$(0, c)$	1	$(0, caaa)$	$(0, d)$	0	$(0, \varepsilon)$
$(0, d)$	1	$(0, dbbbb)$	$(1, a)$	0	$(1, \varepsilon)$
$(1, a)$	1	$(1, aa)$	$(1, c)$	0	$(0, \varepsilon)$
$(1, c)$	1	$(1, cbb)$			

These are eleven transition sites. This is the two-physical-row return-typed compilation of the canonical $m = 3, k = 1$ automaton under the row partition $(0, 1, 0)$. More explicitly, the original four slope modes are $(1, A), (1, M), (0, M), (2, M)$, and the physical source-state and stack-letter roles are encoded by the displayed table. Each source input-0 step removes its top letter and transfers control to the row prescribed by the exposed next tag. Each input-1 step inserts the exact counter debt above the unchanged buried stack, or expands the initial header as prescribed. The typed zero-return adjacency ensures that a newly exposed lower symbol is read in the same source control mode as in the canonical machine. The six accessible physical control/top types are $(0, b), (1, a), (0, a), (1, c), (0, c), (0, d)$. Each input-0 action removes its top letter and exposes the prescribed return type; each of the five input-1 rewrites installs a typed block whose zero-drain return control agrees with that of the replaced symbol, independently of any buried tail. The relation between source stack tags and physical future-exposure types is therefore invariant under every input step. Induction proves equivalence on all finite words, including undefined actions and the final-and-physically-empty acceptance condition. The independent literal table was also compared against the canonical source on all 16,383 binary inputs of length at most 13 and 1,200 longer random inputs, with no mismatch. □

Corollary A.13 (Conditional complete epsilon-free frontier)**.** *Subject to the full mathematical validity of the archived G72–G89 certificate chain and the representation-independent one-state and one-letter lower lemmas for this witness, the epsilon-free Pareto frontier is*

$$\boxed{\mathrm{Pareto}_{\mathrm{rt}}(B_{3,1}) = \{(1, 6), (2, 4), (3, 2)\}.}$$

In particular the exact epsilon-free two-state stack-alphabet threshold is $b_2^{\mathrm{rt}}(B_{3,1}) = 4$.

Proof. The one-control lower bound $b_1 \geq 6$ and a matching one-control realtime typed construction give $(1, 6)$. Proposition A.11 excludes $(2, 3)$, and Proposition A.12 attains $(2, 4)$. The canonical three-control/two-letter source attains $(3, 2)$, and the independent unary-stack impossibility theorem excludes all $(q, 1)$. Upward closure of resource attainability therefore leaves precisely the three listed coordinatewise minimal points. The negative $(2, 3)$ conclusion is *not* transferred to epsilon-enabled machines. □

Remark A.14 (Certificate provenance and reproducibility)**.** *The terminal G88 source-distinct and G89 source-sharing UNSAT programs were replayed and independently enumerated in separate implementations. The G86 cost-two/cost-three scan was independently reproduced in full; G88 produced* 385 *compatible summaries,* 188,089 *physical exceptional input-one options, and the same last* 32 *tables; G89 produced* 7,847 *complete physical zero-rule tables and the same last* 18 *contradictions. The independent* $(2, 4)$ *checker verifies the 11-transition future-exposure invariant without calling the archived source compiler. The G101 checkpoint additionally records successful replay of all 15 predecessor and terminal scripts. Executable copies, logs and digests are retained in an internal research archive and are* not *submitted with this PDF-only article. Reproduced finite counts do not formally certify the necessary-reduction lemmas; the conditional status of Corollary A.13 remains.*

B Certificate provenance, reproducibility and limitations

The canonical calibration was separately checked by a transition-level interpreter on representative (m, k, H) tuples. These finite tests corroborate selected source identities but do not establish the general normalization and compactness theorems, whose mathematical proofs are presented in the main text. The resource decision in Theorem 6.2 is a terminating *conceptual* algorithm; its finite-table search and deterministic-PDA equivalence component may have prohibitive complexity. No explicit uniform small cutoff for the number of ones is claimed.

The finite-refutation format has been independently tested at $(m, k, q, g) = (2, 1, 1, 1)$: 18 tables at $H = 0$, 50 at $H = 1$ and 128 at $H = 2$ were enumerated, and the three-word $H = 2$ packet refutes the final level. The archived G88/G89 two-state/three-letter obstruction for $B_{3,1}$ was replayed successfully together with an independently constructed epsilon-free $(2, 4)$ automaton. The G85–G87 original programs have been re-executed; an independent full G86 constraint scan, a full G88 candidate-generation check, and an independent full G89 zero-table census additionally matched their archival counts and terminal physical witnesses. The exact Pareto consequence nevertheless remains conditional on independent proof-to-code validation of the full G71–G89 case coverage, especially G75/G78, in Section A; the G72/G74 mathematical arguments have been reproduced. The outstanding questions include obtaining useful complexity or witness-size bounds, classifying further languages with effective zero-drain calibrations, exhibiting practical finite-horizon certificates for nontrivial (q, g) pairs, and comparing this criterion with related equivalence and descriptional-complexity literature. In particular the epsilon-enabled resource-existence problem and the complete numerical Pareto frontier of $B_{m,k}$ remain open. No statement here conflates an epsilon-free NO certificate with an unrestricted epsilon-DPDA impossibility.

Statements and Declarations

Author contributions. Alp Eren Bütün: conceptualization, methodology, formal analysis, investigation, software, validation, writing—original draft, and writing—review and editing.

Funding. No external funding was received for this research.

Competing interests. The author declares no relevant financial or non-financial competing interests.

Data availability. No empirical or personal dataset was used. Definitions, theorem statements, and mathematical proofs are included in this article.

Code availability. No executable code or verification supplement accompanies this submission. The general epsilon-free resource-existence decidability theorem is established by the mathematical proof in the article and does not require the finite-search case study. The $B_{3,1}$ case study reports internally checked finite enumerations and remains conditional on the necessary-reduction and coverage assumptions stated in the text. Its underlying programs are not included, so the enumerations cannot be independently rerun from the submitted PDF alone.

Ethics approval and consent to participate. Not applicable.

Consent for publication. Not applicable.

References

[1] A. E. Bütün, "A Binary Stack-Alphabet Hierarchy for Deterministic Pushdown Automata with ε-Moves: Completion Slopes, Up-Modes, and a Product Lower Bound," Zenodo (2026). DOI: 10.5281/zenodo.23445469.

[2] A. E. Bütün, "Exact Pareto Corners and Uniform State–Stack Tradeoffs in a Binary Deterministic Pushdown Hierarchy," related research manuscript (2026).

[3] P. Jančar, "Equivalence of pushdown automata via first-order grammars," *Journal of Computer and System Sciences* **115** (2021), 86–112. doi:10.1016/j.jcss.2020.07.004.

[4] G. Pighizzini, "Push complexity: Optimal bounds and decidability," *Theoretical Computer Science* **1071** (2026), Article 115839. doi:10.1016/j.tcs.2026.115839.

[5] G. Sénizergues, "$L(A) = L(B)$? Decidability results from complete formal systems," *Theoretical Computer Science* **251** (2001), 1–166. doi:10.1016/S0304-3975(00)00285-1.

[6] J. Goldstine, J. K. Price and D. Wotschke, "On reducing the number of stack symbols in a PDA," *Mathematical Systems Theory* **26** (1993), 313–326. doi:10.1007/BF01189852.

[7] L. Kiss and B. Rovan, "Descriptional complexity of push down automata," *Information Technologies–Applications and Theory (ITAT 2020)*, CEUR Workshop Proceedings **2718** (2020), 59–66.